News

Table of Contents

Version 0.7.0 . . . 1
Version 0.6.1 . . . 5
Version 0.6 . . . 7
Version 0.5.7 . . . 8
Version 0.5.6 (fix bugs from 0.5.5) . . . 9
Version 0.5.4 . . . 10
Version 0.5.3 . . . 11
Version 0.5.2 . . . 12
Version 0.5.1 . . . 13
Version 0.5.0 . . . 15
Version 0.4.2 . . . 16
Version 0.4.1 . . . 17
Version 0.4.0 . . . 19
Version 0.3.9 . . . 21

Version 0.7.0

- Grails 7, Gradle 8.4, Java 24, Groovy 4.0.28 ...
- Improve Performances, start transmitting page earlier and should consume less memory per page
- Split `taack-ui` into `taack-ui`, `taack-ui-jdbc`, `taack-ui-pdf`, `taack-ui-ssh`, `taack-ui-search`, `taack-ui-asciidoc`. Minimal server using only `taack-ui` should be a Jar around 70 MB in size
- Contextual Menus for fields and classes (See Contextual Menu Code Sample)
- Notification helper: allow to mark unread rows on a table via `setLastReadingDate` filter setter. (See Create filter marking unread entries)
- SolrSearch changes: objects context have to be registered in a generic manner (See Register how an object will be shown)
- New Diagram personalisation through builders easing mail and pdf diagram integration (See Diagram Builder)
- Asciidoc Editor with copy/past capabilities (See Asciidoc Editor)
- Improve Mail and PDF Diagram output

Contextual Menu Code Sample

```
TaackUiService.registerContextualMenuClosure(    ①
    new Lead().leadStatus_,
    new UiMenuSpecifier().ui { Long id ->        ②
        label 'Change Status'
        Lead.read(id)?.leadStatus?.transitionsTo()?.each {
            menu it.toString(),                  ③
                CrmController.&changeLeadStatus as MC,
                [id: id, leadStatus: it.toString()]
    }
})
```

① Register a contextual menu for a field

② Use Regular MenuSpecifier closure (as for any menus)

[screenshot contextual menu on field] | *screenshot-contextual-menu-on-field.webp*

Figure 1. Result of a right click on Lead Status

Create filter marking unread entries

```
TaackFilter.FilterBuilder tf = taackFilterService.getBuilder(Item)
        .setMaxNumberOfLine(10)
        .setSortOrder(TaackFilter.Order.DESC, i.dateCreated_)

tf.setLastReadingDate(lastAccessDateFromUser)    ①
```

① Newest objects, will be bold, See Unread rows where object `lastUpdated` > `lastAccessDateFromUser`

[screenshot notif] | *screenshot-notif.webp*

Figure 2. Unread rows where object `lastUpdated` > `lastAccessDateFromUser`

Register how an object will be shown

```
TaackGormClassRegisterService.register(
        new TaackGormClass(ItemRange.class).builder
                .setShowMethod(Bp2Controller.&showRange as MethodClosure)   ①
                .build(),
        new TaackGormClass(Item.class).builder
                .setShowMethod(Bp2Controller.&showItem as MethodClosure)    ①
                .setShowLabel({ Long id ->                                  ②
                    def i = Item.read(id)
                    return "${i.name} ${i.ref ?: ''} ($id)"
                })
                .setTypeLabel({ Long id ->                                  ③
                    return TaackUiService.tr('default.item.label')
                }).build()
)
```

① How to Show the Object

② Custom Label

③ Type for Faceting

Diagram Builder

```
new UiDiagramSpecifier().ui {
    bar {
        labels 'xLabel1', 'xLabel2', 'xLabel3', 'xLabel4'
        dataset 'dataset1', 1.0, 2.0, 3.0, 4.0
        dataset 'dataset2', 6.0, 1.0, 2.0, 4.0

        option DiagramOption.builder
            .setClickAction(Rma2Controller.&editClaim as MethodClosure)
            // "diagramAction" was moved to here
            .showDataCount()
            // The numbers are hidden by default. Use this option if need to show
            .hideLegend()
            // The legends are shown by default. Use this option if need to hide
            .setResolution(DiagramOption.DiagramResolution.DEFAULT_4K)
            // Default resolution is [SVG: 960x480, PNG: 2160x1080].
            // Allow to set to 540p, 720p, 1080p, 2K, 1440p, 4K
            .setKeyColors(Color.RED, Color.BLUE)
            // Allow to self-define the colors for each dataset
            // (If not set, there are by default 7 colors)
            .setTitle("Test Chart") // Diagram title
            .build()
    }
}
```

Asciidoc Editor

```
//...
EditorOption.EditorOptionBuilder editor = EditorOption.getBuilder()
editor.addSpanRegexes(TaackBaseAsciidocSpans.spans)          ①
editor.addSpanRegexes(TaackAsciidocTable.spans)              ①
editor.addSpanRegexes(TaackAsciidocPlantUML.spans)           ①
editor.uploadFileAction(this.&dropEditor as MC, cmsPage.id) ②
fieldEditor cmsPage.bodyContent_, editor.build()             ③
innerFormAction this.&previewBody as MC                      ④
//...

@Transactional
def dropEditor() {                                           ②
    CmsPage page = CmsPage.get(params.long('cmsPage'))
    params.remove('cmsPage')
    String past = params.get('onpaste')
    if (past) {                                              ⑤
        render taackAsciidocService.convertFromHtml(past)
    } else {                                                 ⑥
        final List<MultipartFile> mfl = (request as MultipartHttpServletRequest
).getFiles('filePath')
        final mf = mfl.first()

        if ([AttachmentContentType.SHEET_ODS.mimeType,
             AttachmentContentType.LO_TEXT.mimeType
            ].contains(mf.contentType)) {                    ⑦
            render taackAsciidocService.convert(page, mf.inputStream)
        } else {                                             ⑧
            CmsImage cmsImage = taackSaveService.save(CmsImage)
            if (cmsImage && page) {
                cmsImage.cmsPage = page
            }
            render "image::${cmsImage.originalName}[]"
        }
    }
}
```

① Add syntax spans extensions (basically a regex and a class to colorise the code)

② Callback when a file is drop, or a past from another page or application is done

③ The field in the form

④ Optional Preview Action

⑤ Past a html content. Content will be converted into Asciidoc

⑥ Drop file part

⑦ The file is an ODS or Writer file, it will be converted into Asciidoc, image will be imported

⑧ The file is an image, it will be saved into the CMS.



taackAsciidocService uses `TaackEditorService` service from `taack-ui-asciidoc`, see form DSL

Version 0.6.1

- Diagram Action (see Diagram Action Usage)
- Fullscreen modal
- Static helpers (see Static Helpers Usage)

Diagram Action Usage

```
new UiDiagramSpecifier().ui {
    bar {
        labels 'date1', 'date2', 'date3'
        dataset 'Stuff1', 3.0, 4.0, 5.0

        diagramAction this.&clickDiagram as MC, id, ①
        [optionalParam: 'value']                    ②
    }
}

def clickDiagram() {
    println(params)
    // [id: 123456, dataset: "Stuff1",
    // optionalParam: "value"                       ③
    // x: "date1", y: "3.0"]
}
```

① Diagram Action

② Can pass map

③ Action params contain **label**, **value**, **dataset name** and **map**

Static Helpers Usage

```
import org.codehaus.groovy.runtime.MethodClosure as MC

import static taack.ui.TaackUi.createMenu           ①

@Override
UiMenuSpecifier editorCreate() {
    createMenu {                                    ②
        menu this.&createFromTemplate as MC
        menuIcon
            CollaboraIcons.WRITER,
            this.&createFromTemplate as MC,
            [collaboraApp: CollaboraApp.WRITER]
    }
}
```

① Static Import

② `createMenu` static call, shortcut for `new UiMenuSpecifier().ui`, other shortcuts include `createModal`, returning `UiBlockSpecifier`

▶ https://www.youtube.com/watch?v=wF323zHFa94 *(YouTube video)*

Version 0.6

- Load animation
- Diagram zoom / scroll
- Diagram tooltips
- Table Multiselect (see doc/DSLs/filter-table-dsl.pdf)
- Tabs lazy loading
- Improve pagination
- Optimize drawing, avoiding unnecessary draw
- Avoid case where blocks were drawn twice
- Improve search layout
- Allow Big decimal on tables, shown in user's locale
- WiP: Simple Asciidoc WYSIWYG Editor

Version 0.5.7

- Clean up `show` DSL code, deprecates passing object parameter to `UiShowSpecifier().ui`
- Initial Asciidoctor WYSIWYG editor
  - Support for Drag and Drop images and files
- More diagram DSLs
  - timeSeries
  - areaChart
  - bubbleChart
- Security
  - Sanitize displayed information by default (use `fieldRaw` to avoid sanitizing)
  - Check access on all entry points
- Bug fixes and dependencies bump



Replace `field <html code>` by `fieldRaw <html code>`

Version 0.5.6 (fix bugs from 0.5.5)

- reduce Solr and tika dependency scoping
- Show table sortable column and sorting direction (see Column headers show sorting directions)
- Block menus are now refreshed like blocks
- For complexe layout, code can be reused easily. We can now keep variables that contextualize the layout easily (see How to keep the context when clicking on a table)

[screenshot news sorting] | *screenshot-news-sorting.webp*

Figure 3. Column headers show sorting directions

How to keep the context when clicking on a table

```
def showPart(PlmFreeCadPart part, Long partVersion, Boolean isHistory) {①
    taackUiService.show(
            plmFreeCadUiService.buildFreeCadPartBlockShow(
                    part, partVersion, false, isHistory),              ②
            buildMenu(),
            "isHistory")                                               ③
}
```

① `isHistory` is an action parameter

② `isHistory` is used when drawing the block; we need to retransmit it to draw the exact same block layout, by keeping the context

③ `isHistory` key is passed as the last `taackUiService.show` argument. You can put many keys to keep.

Version 0.5.4

- Rework diagram DSL (See doc/DSLs/diagram-dsl.adoc)

Version 0.5.3

- Fix form checkbox
- Allow alias in **TQL** for formula columns
- Code cleanup and increment dependency versions

Version 0.5.2

- JDBC client is now also an AsciidoctorJ extension
- Add getters to JDBC accessible domain fields
- Add DSL TQL and TDL (Taack Display Language) for describing how to display queried data (either table or barchart)
- Restore manual labeling on menus
- More on diagram DSL (Thanks Chong and ZhenQing)
- Better customisation

TQL and TDL (Taack Display Language)

```
select
    u.rawImg,
    u.username,
    u.manager.username
from User u
where u.dateCreated > '2024-01-01' and u.manager.username = 'admin';
--
table rawImg as "Pic", username as "Name", manager as "Manager"
```

[news table] | *news-table.webp*

Figure 4. Results

Version 0.5.1

- Replacement of TaackPlugin, app module registers itself independently
- Remove Charts DSL
- Fix Diagram DSL, Replacement of Charts: Diagrams
- Allow Diagrams into PDF (See PDF containing diagrams and Stacked Bar Diagram)

Replacement of TaackPlugin

```
@PostConstruct
void init() {
    TaackUiEnablerService.securityClosure(
        this.&securityClosure,
        CrewController.&editUser as MC,
        CrewController.&saveUser as MC)
    TaackAppRegisterService.register(
        new TaackApp(
            CrewController.&index as MC,                    ①
            new String(
                this.class
                    .getResourceAsStream("/crew/crew.svg")  ②
                    .readAllBytes()
            )
        )
    )
}
```

① Entry Point

② Icon

Replacement of Charts: Diagrams

```
private static UiDiagramSpecifier d1() {
    new UiDiagramSpecifier().ui {
        bar(["T1", "T2", "T3", "T4"] as List<String>, false, {
            dataset 'Truc1', [1.0, 2.0, 1.0, 4.0]
            dataset 'Truc2', [2.0, 0.1, 1.0, 0.0]
            dataset 'Truc3', [2.0, 0.1, 1.0, 1.0]
        }, DiagramTypeSpec.HeightWidthRadio.ONE)
    }
}
```

PDF containing diagrams

```
printableBody {
    diagram(d1(), BlockSpec.Width.HALF)
    diagram(d2(), BlockSpec.Width.HALF)
```

```
}
```

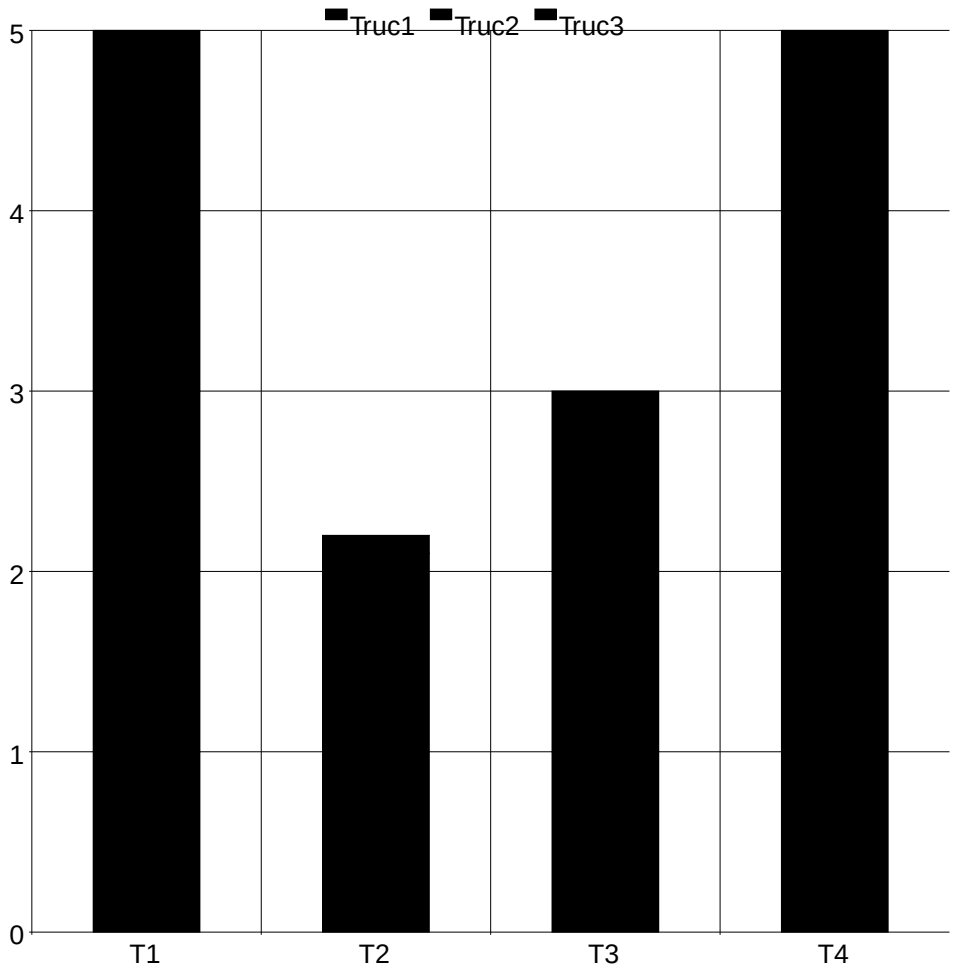


Figure 5. Stacked Bar Diagram

Version 0.5.0

slide::[fn=slideshow-whatsnew050-en]

Version 0.4.2

To be released… this version should come with some nice changes (breaking some old code sometime)

- Improve DSL hierarchy
  - hidden fields on top only for readability
  - no redundant parameter passing in form
  - no redundant parameter passing in filter
  - filterField only under section only
  - form top level field only on header
- hook for form fields to display M2M nicely
- hook to register typical object filter
- Improve restore state
- Fix table grouping / trees with paginate
- TBD

Version 0.4.1

- Merge search menu, icon menu, and language Menu, see Updated Menus layout
- Keep some params … (lang, subsidiary, stock, others …) via menu DSL
  - Move supported Language into menus (from plugin declaration), see Menus layout code
- Allow debugging Kotlin JS code, see Kotlin JS Debug HowTo
- Fix file path when updating. The same way as for O2M, with preview
- Improve restore state
- Test mac runtime and devel cold auto-restart
- Solr indexField auto-labeling, see New Solr DSL Simplification (no more labeling needed)

[screenshot news menu 0.4.1] | *screenshot-news-menu-0.4.1.webp*

Figure 6. Updated Menus layout

Menus layout code

```
private UiMenuSpecifier buildMenu(String q = null) {
    new UiMenuSpecifier().ui {
        menu CrewController.&index as MC
        menu CrewController.&listRoles as MC
        menu CrewController.&hierarchy as MC
        menuIcon ActionIcon.CONFIG_USER, this.&editUser as MC
        menuIcon ActionIcon.EXPORT_PDF, this.&downloadBinPdf as MC
        menuSearch this.&search as MethodClosure, q
        menuOptions(SupportedLanguage.fromContext())            ①
    }
}
```

① Language choice is on the right of the searchbar, and other enums can be added

Kotlin JS Debug HowTo

```
$ cd infra/browser/client                              ①
$ ./gradlew browserDevelopmentRun                      ②
$ vi intranet/server/grails-app/conf/application.yml   ③
# Uncomment line bellow
# client.js.path: 'http://localhost:8080/client.js'

# Then your browser should show Kotlin code !
```

① Move to client folder where JS code is generated

② Launch a server serving client.js and client.js.map …

③ edit your intranet `application.yml` file

New Solr DSL Simplification (no more labeling needed)

```
@PostConstruct
private void init() {
    taackSearchService.registerSolrSpecifier(this,
            new SolrSpecifier(User,
                CrewController.&showUserFromSearch as MethodClosure,
                this.&labeling as MethodClosure, { User u ->
        u ?= new User()
        indexField SolrFieldType.TXT_NO_ACCENT, u.username_
        indexField SolrFieldType.TXT_GENERAL, u.username_
        indexField SolrFieldType.TXT_NO_ACCENT, u.firstName_
        indexField SolrFieldType.TXT_NO_ACCENT, u.lastName_
        indexField SolrFieldType.POINT_STRING, "mainSubsidiary", true, u.subsidiary
?.toString()
        indexField SolrFieldType.POINT_STRING, "businessUnit", true, u.businessUnit
?.toString()
        indexField SolrFieldType.DATE, 0.5f, true, u.dateCreated_
        indexField SolrFieldType.POINT_STRING, "userCreated", 0.5f, true, u
.userCreated?.username
    }))
}
```

Version 0.4.0

- No more `paginate` in tables. See New `iterate` usage
- No `list`, but an `iterate` taking a closure as parameter, with a builder pattern approach to pass args
- Menu are auto labeled now (use `lang=test` in url to translate menus). See New `menu` code
- No more isAjax parameter in tables ... See New `rowAction` code
- change rowLink into rowAction [i18n_isAjax]
- No label needed on rowAction in tables. See New `rowAction` code
- No more ajaxBlock required for tables, forms, tableFilters
- formAction has no more isAjax parameter
- formAction has no more mandatory i18n parameter
- form has no more mandatory i18n parameter, i18n is based on current action name
- block action has no more mandatory i18n parameter, i18n is based on target action
- block action has no more mandatory isAjax parameter

New `iterate` usage

```
iterate(taackFilterService.getBuilder(Role)                         ①
        .setMaxNumberOfLine(20)                                     ②
        .setSortOrder(TaackFilter.Order.DESC, u.authority_)         ③
        .build()) { Role r, Long counter ->
            row {
                rowColumn {
                    rowField r.authority
                    if (hasSelect)
                        rowAction
                            ActionIcon.SELECT * IconStyle.SCALE_DOWN,
                            CrewController.&selectRole as MC
                            r.id                                    ④
                }
            }
        }
```

① iterate

② Specifying max is enough to trigger paginate if more lines

③ Replace the old inefficient pattern to describe initial sort and order

④ No more i18n and isAjax parameter

New `menu` code

```
private UiMenuSpecifier buildMenu(String q = null) {
    UiMenuSpecifier m = new UiMenuSpecifier()
```

```
    m.ui {
        menu CrewController.&index as MC        ①
        menu CrewController.&listRoles as MC
        menu CrewController.&hierarchy as MC
        menuSearch this.&search as MethodClosure, q
    }
    m
}
```

① No i18n parameter

New `rowAction` *code*

```
if (hasActions) {
    rowColumn {
        rowAction ActionIcon.EDIT * IconStyle.SCALE_DOWN, this.&roleForm as MC, r.id
①
    }
}
```

① No i18n parameter, no last `isAjax` parameter

Version 0.3.9

No updates since too long, hibernation is coming to an end. This version offer:

- Grails 6.2.0
- Groovy 3.0.21
- Bumping Various deps … (See Changelog)